A.L.T.R.U.S – Adaptive Life-support & Therapeutic Robotic Unit System

Project ID: 25-26j-437

Project Proposal

Mallawaarachchi U.V.N – IT22092274

Supervised by: Ms. Hansi De Silva

Co-supervised by Mr. Ashvinda Iddamalgoda

B.Sc. (Hons) Degree in Information Technology

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

July 2025

DECLARATION

We are confirming that the work given in this proposal is one of our own volition. The same work has not been given, entirely or partially, for academic credit at any other university or institution. We have also made sure that all sources of information and ideas from others have been clearly cited and appropriately acknowledged in the document.

Name	Student ID	Signature
Mallawaarachchi U.V.N	IT22092274	

The above candidates are researching the undergraduate Dissertation under my supervision.

Name of supervisor: Ms. Hansi De Silva

Name of co-supervisor: Mr. Ashvinda Iddamalgoda

Signature of supervisor: Date: 2025/08/

Signature of co-supervisor: Date: 2025/08/

ABSTRACT

This article presents A.L.T.R.U.S (Adaptive Life-support & Therapeutic Robotic Unit System), a context-aware, modular assistive robot designed for safe, smart, and empathetic behavior in dynamic home scenarios. The system integrates four core software frameworks within a ROS2-based architecture to overcome the majority of shortcomings of current assistive robotics. An adaptive navigation module initially combines static SLAM with dynamic real-time LIDAR-based updates according to RSO2 logic to enable dynamic replanning of paths in response to environmental change—remapping is not needed. Voice and hand gesture control is integrated directly into the navigation layer so that users can call, pause, or drive the robot by natural voice commands or by a hand wave, enabling much enhanced accessibility for the disabled and elderly. Second, an environmental and health monitoring system in real-time combines data from environmental sensors (MQ-2, DHT11) and vital sign sensors (MPU6050, MAX30100) to detect emergencies like gas leaks, fever, or falls and provide automatic alerts via Firebase and Twilio. Third, an emotion-perceiving HRI module detects facial expressions and voice tone with OpenCV and pre-trained models (RAVDESS, FER2013), providing empathetic responses through a chatbot based on Dialogflow. Fourth, a fault-tolerant middleware offers system robustness through auto-recovery, intent arbitration, plugin update security, and blockchain log-based recording with Hyperledger Fabric for trust and transparency. Based on a modular ROS2 foundation, A.L.T.R.U.S brings together adaptive navigation, proactive healthcare, emotional intelligence, and robust software infrastructure in a unified, edge-deployable platform. This student innovation stretches the limits of assistive robotics by combining autonomy, safety, empathy, and security, which is ideal for smart homes, elderly care, and independent living support.

Keywords:

Adaptive Navigation, Dynamic SLAM, Human-Robot Interaction, Voice Command, Gesture Recognition, Emotion Recognition, Assistive Robotics, ROS2, Sensor Fusion, Blockchain, Proactive Healthcare

TABLE OF CONTENTS

DECLARATION2

ABSTRACT3

TABLE OF CONTENTS4

LIST OF FIGURES6

LIST OF TABLES7

LIST OF ABBREVIATIONS8

1. INTRODUCTION9

1.1 BACKGROUND9

1.2 LITERATURE SURVEY13

1.2.1 Middleware Architectures in Robotics13

1.2.2 Intent Recognition Systems13

1.2.3 Fault-Tolerant Robotics14

1.2.4 Blockchain Integration in Robotics14

1.3 RESEARCH GAP15

1.4 RESEARCH PROBLEM17

2. RESEARCH OBJECTIVES18

3. METHODOLOGY19

3.1 System Design20

3.2 Evaluation Metrics22

3.3. Expected Results and Impact22

3.6 Commercialization23

4. PROJECT REQUIREMENTS24

4.1 Functional Requirements**Error! Bookmark not defined.**

4.2 Non-functional Requirements**Error! Bookmark not defined.**

4.3 User Requirements**Error! Bookmark not defined.**

5. BUDGET DETAILS27

6. CONCLUSION28

References29

APPENDICES30

Appendix 1. Work Breakdown Structure (WBS)30

Appendix 2. Gantt Chart31

31

Appendix 3. Online Survey31

LIST OF FIGURES

Figure 1.Survey results on importance of system reliability in assistive robots12

Figure 2. Main concerns about robotic system failures12

Figure 3.Preference for blockchain-backed logs and automated fault recovery12

Figure 4.3.1 System Overview Diagram19

Figure 5.3.2 Middleware architecture diagram showing Intent Engine, Fault Recovery, Blockchain Logger, Plugin Manager, Event Bus20

LIST OF TABLES

Table 1.2 Comparative analysis highlighting the novelty of the proposed system15
Table 2.1 Budget for the proposed system27

LIST OF ABBREVIATIONS

ABBREVIATION	DESCRIPTION
SLAM	Simultaneous Localization and Mapping
ROS2	Robot Operating System 2
LIDAR	Light Detection and Ranging
NLP	Natural Language Processing
HRI	Human-Robot Interaction
STT	Speech-to-test
GUI	Graphical User Interface
Gazebo	Robot Simulation Environment
FTR	Fault Tolerance & Recovery
IE	Intent Engine
RTEB	Real-time Event bus
BCL	Blockchain Logger
CLI	Command Line Interfaces
API	Application Programming Interface

1. INTRODUCTION

1.1 BACKGROUND

The home care future lies in smart, empathetic, and learning-enabled robots—platforms that not only support and move but also understand, respond, and learn with users. To meet the growing demand for affordable, personalized, and decentralized care, this project proposes a next-generation assistive robot for deployment within dynamic home environments.

Drawing inspiration from emotionally intelligent robots such as Baymax, our framework combines four fundamental software building blocks in one coherent, ROS2-based system:

Voice-Based Command & Adaptive Navigation:
This module combines RPLIDAR-based SLAM with dynamic replanning of paths using reinforcement learning to enable safe navigation around evolving home settings. The robot re-routes in real time when objects are encountered or furniture is shifted by responding to voice instructions, enabling non-stop autonomy even in dynamic settings.

Real-Time Health & Environmental Monitoring
The framework combines sensor data from wearables and the environment (e.g., heart rate, temperature, gas concentrations) to monitor user well-being in real time. Upon emergencies such as falls, fever, or leaks of gas, the framework triggers automatically generated alerts via cloud services like Firebase and Twilio, enabling remote caregivers to respond immediately. This feature offers proactive monitoring and timely intervention.

Emotion Detection & Empathetic Behavior Generation:
With the implementation of technologies such as OpenCV and TensorFlow, the system reads facial expressions and voice tone to identify emotions like sadness, worry, or suffering. A chatbot interface generates empathetic responses (e.g., "You look upset. Do you want to talk?"), with users empowered to converse nonverbally through gesture recognition. The multimodal approach enhances natural interaction and emotional response, and the robot is that much more intuitive and useful for technical or elderly users.

Fault-Tolerant Middleware with Secure Plugin Updates:
Built on ROS2, the system possesses a fault-tolerant, modular middleware that supervises all components. It provides support for intent-based behavior arbitration (resolution of conflicting commands) and secure, blockchain-logged plugin updates, ensuring reliability and trust for mission-critical healthcare functions. This architecture enables strong runtime recovery, dynamic skill composition, and secure communication, making the system strong and scalable to evolving needs.

This merged system addresses key limitations in current assistive robots, including inflexible navigation, mechanical interaction, health insensitivity, and vulnerability to breakdowns. Through incorporating adaptive navigation, natural human-robot interaction, real-time monitoring of health, and fault-tolerant middleware, the resulting system provides context awareness, emotional intelligence, and self-sustenance for practical deployment. It transforms traditional assistive robots into intelligent companions that can dynamically adapt to changing environments while offering natural, multimodal control interfaces—all within a modular, scalable framework like ROS2 for real-world assistive applications.

Assistive robotics has emerged as a transformative technology with the potential to improve the quality of life for individuals requiring support in daily activities, rehabilitation, and healthcare management. The global aging population and increasing prevalence of chronic diseases have accelerated the demand for intelligent robotic systems that can seamlessly integrate into human-centric environments. These robots are designed to provide assistance in tasks ranging from basic mobility support to complex therapeutic interventions, thereby reducing dependency on human caregivers and improving autonomy for users.

The core challenge in developing such systems lies in their ability to operate reliably in dynamic, unpredictable environments. Assistive robots must interpret human intentions accurately, adapt to changes in the environment, and maintain operational integrity even in the face of hardware or software failures. Middleware architectures play a pivotal role in enabling this functionality by providing standardized frameworks for communication, coordination, and integration among heterogeneous robotic modules. Effective middleware allows modular components—such as sensors, actuators, and AI modules—to interact seamlessly, facilitating scalability, adaptability, and interoperability.

Moreover, modern assistive robots are increasingly data-driven, relying on continuous feedback from sensory inputs and decision-making algorithms. This necessitates a robust system capable of managing real-time data streams, processing complex computations, and ensuring secure storage and transmission of sensitive information. Blockchain technology, with its decentralized and tamper-proof characteristics, presents a promising avenue for maintaining the integrity and traceability of robotic operations, thereby enhancing trustworthiness and reliability.

1.2 Problem Context

Despite significant advancements, current modular robotic systems often face critical limitations that hinder their widespread adoption. One of the primary challenges is uncoordinated fault management. Many existing middleware solutions are reactive, addressing failures only after they occur, rather than implementing proactive mechanisms that anticipate and mitigate potential errors. This reactive approach can lead to system

downtime, reduced reliability, and compromised safety in environments where human-robot interaction is frequent and critical.

Another major issue is the limited integration of intent-driven decision-making. While research in human action and intent recognition has made considerable progress through deep learning models such as convolutional neural networks (CNNs) and recurrent neural networks (RNNs), these capabilities are rarely incorporated directly into middleware frameworks. Consequently, robots may fail to interpret human commands accurately or respond in a timely manner, reducing effectiveness and user satisfaction.

Additionally, the security and integrity of operational data remain significant concerns. Traditional logging mechanisms are often centralized and susceptible to tampering, which can compromise system accountability and reliability. In healthcare and assistive contexts, ensuring data security is paramount, as any breach or manipulation can have severe consequences for users and caregivers.

1.3 Significance of the Study

This research aims to address the aforementioned challenges by developing a middleware architecture that is simultaneously fault-tolerant, intent-driven, and secure. By integrating a proactive fault detection and recovery mechanism, the system will be capable of identifying potential failures before they escalate, minimizing downtime and enhancing operational reliability.

The incorporation of a deep learning-based Intent Engine will enable robots to interpret and respond to human actions and commands accurately, thereby improving user interaction and overall system responsiveness. Furthermore, the integration of blockchain technology will ensure secure, tamper-proof logging of all system operations, providing transparency, accountability, and traceability.

The expected outcome of this study is a robust, modular middleware framework that can seamlessly integrate with diverse robotic components, offering scalability and adaptability in a wide range of assistive scenarios. By bridging gaps in fault management, intent-driven control, and operational security, this research not only advances the state-of-the-art in assistive robotics but also sets a foundation for future innovations that can significantly improve the autonomy, safety, and quality of life for users in healthcare and rehabilitation environments.

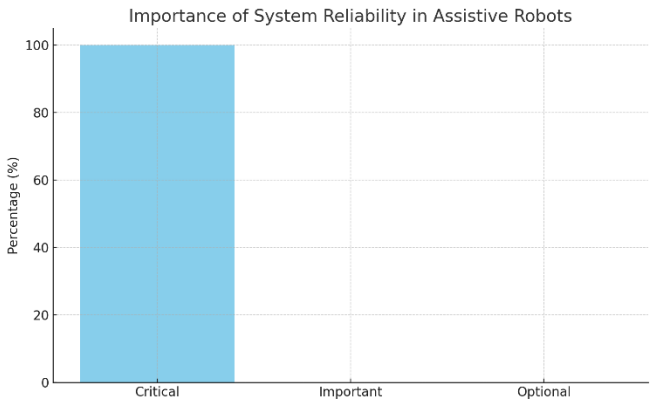


Figure 1.Survey results on importance of system reliability in assistive robots

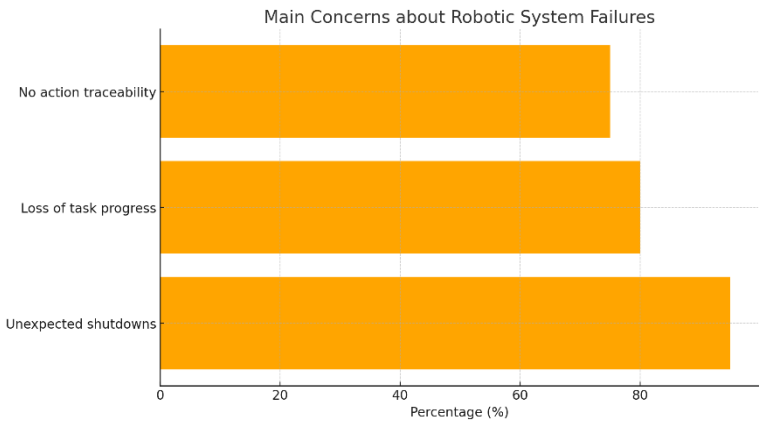


Figure 2. Main concerns about robotic system failures

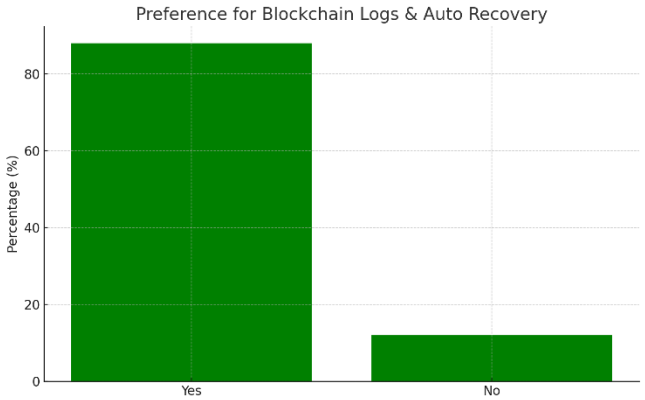


Figure 3.Preference for blockchain-backed logs and automated fault recovery

1.2 LITERATURE SURVEY

1.2.1 Middleware Architectures in Robotics

Middleware serves as the backbone for robotic systems, providing essential services that facilitate communication, coordination, and integration among various robotic components. Existing middleware frameworks such as ROS (Robot Operating System), YARP (Yet Another Robot Platform), and OROCOS (Open Robot Control Software) have significantly contributed to advancing modular robotic systems. ROS, for instance, offers a flexible and standardized platform that supports message passing, service calls, and modular package management, enabling developers to build complex robotic applications efficiently. YARP emphasizes real-time communication and device abstraction, while OROCOS specializes in real-time control for industrial applications.

Despite their strengths, these middleware solutions exhibit notable limitations in the context of modern assistive robotics. While they excel at facilitating inter-module communication, they generally lack comprehensive fault tolerance mechanisms, which are crucial for ensuring reliability in human-centric environments. Additionally, they do not provide integrated secure logging capabilities, leaving sensitive operational data vulnerable to tampering or loss. The absence of holistic system monitoring and fault management makes these platforms less suitable for applications that require high reliability, security, and adaptability, such as healthcare and rehabilitation robots.

1.2.2 Intent Recognition Systems

Intent recognition is a critical aspect of human-robot interaction, enabling robots to interpret and predict human actions and respond appropriately. Deep learning techniques, particularly Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), have demonstrated significant potential in modeling temporal and spatial patterns inherent in human behavior. CNNs are adept at processing visual and spatial data, such as gestures or postures, while RNNs excel at modeling sequential and temporal dependencies, making them suitable for understanding continuous human actions.

Although these approaches have achieved high accuracy in research settings, their integration into middleware remains limited. Existing robotic middleware rarely incorporates real-time intent recognition, which restricts the robot's ability to proactively respond to user commands or adapt to dynamic environments. The challenge lies in developing an efficient and scalable framework that can seamlessly incorporate intent recognition models, enabling modular robots to interpret human intentions in real-time and act in a coordinated manner across multiple modules.

1.2.3 Fault-Tolerant Robotics

Fault tolerance is essential for ensuring uninterrupted and safe operation in assistive robotics. Traditional approaches, including redundancy, checkpointing, and recovery mechanisms, are often reactive; they detect and address faults after they occur. While these methods can restore functionality, they may result in downtime, degraded performance, or safety risks, especially in sensitive human-robot interaction scenarios.

Recent research highlights the importance of proactive fault detection, which anticipates potential failures and triggers preemptive corrective actions. Incorporating proactive fault-tolerant mechanisms within middleware can enhance overall system reliability, minimize operational disruptions, and improve user trust. By monitoring system health metrics, predicting potential failures, and implementing automatic recovery strategies, middleware can ensure that assistive robots operate safely and efficiently in dynamic, unpredictable environments.

1.2.4 Blockchain Integration in Robotics

Blockchain technology has emerged as a promising solution for enhancing the security and integrity of robotic systems. By providing decentralized, tamper-proof logging and secure communication protocols, blockchain ensures that operational data remains transparent and verifiable. This is particularly relevant for assistive robots, where maintaining the authenticity and integrity of system logs is critical for safety, accountability, and regulatory compliance.

Although blockchain has been explored in multi-agent systems and industrial IoT applications, its integration into assistive robotic middleware is still in the early stages. Existing studies primarily focus on data provenance, auditability, and secure transactions, with limited exploration of real-time performance and scalability in robotics applications. There is a clear opportunity to develop middleware that leverages blockchain to provide secure, traceable logging while maintaining high efficiency and responsiveness, thereby enhancing both operational reliability and user trust in assistive robotic systems.

1.3 RESEARCH GAP

Despite significant advancements in robotics middleware, intent recognition, and fault-tolerant systems, several critical gaps persist that hinder the development of fully reliable and secure assistive robotic systems. These gaps underscore the need for a holistic approach that integrates fault tolerance, intent-driven decision-making, and secure operational logging within a single middleware framework.

Feature	**ROS2/DDS**	**Orocos**	**Industrial FTR**	**Blockchain IoT**	**Proposed System**
Fault Recovery	Limited	Limited	Hardware-specific	No	Yes
Intent Recognition	No	No	No	No	Yes
Blockchain Logging	No	No	No	Yes	Yes
Modular Plugin Support	Yes	Yes	No	Limited	Yes
Assistive Robotics Focus	No	No	No	No	Yes

Table 1.2 Comparative analysis highlighting the novelty of the proposed system

3.1 Middleware Integration

Existing middleware platforms such as ROS, YARP, and OROCOS provide foundational communication frameworks and modular integration capabilities. They allow different robotic modules, including sensors, actuators, and control algorithms, to interact effectively. However, these platforms typically lack comprehensive fault-tolerance mechanisms and are not designed to proactively handle system failures. The absence of integrated fault management can result in degraded performance or unexpected system crashes, especially in complex, multi-module environments. There is a clear need for middleware that can not only facilitate module communication but also provide robust fault detection, recovery, and coordination to maintain reliable operations across all integrated components.

3.2 Logging Security

Most existing robotic systems rely on standard logging mechanisms that record operational data in centralized storage systems. While functional, these approaches are vulnerable to tampering, loss, or unauthorized access. In assistive robotics, where user safety and accountability are paramount, compromised logging can lead to safety risks, lack of traceability, and reduced trust in robotic interventions. The gap here is the lack of secure, tamper-proof logging integrated within the middleware. Implementing blockchain-backed logging can address this issue by providing a decentralized, immutable ledger that ensures every action, sensor reading, and module interaction is securely recorded and verifiable in real-time.

3.3 Intent Integration

Intent recognition, often implemented using deep learning techniques like CNNs and RNNs, has been widely explored in robotics research. These systems can predict user actions or commands, enabling more natural and effective human-robot interaction. However, current middleware architectures rarely integrate these models directly, limiting the robot's ability to respond dynamically and proactively to human intentions. Without seamless integration, intent recognition modules operate in isolation, preventing real-time decision-making across the robotic system. There is a need for middleware that fully integrates an Intent Engine, allowing all modules to access and respond to recognized intents efficiently, ensuring cohesive, context-aware behavior.

3.4 Summary Table of Research Gaps

Aspect	**Existing Solutions**	**Limitation**	**Proposed Solution**
Middleware Integration	ROS, YARP, OROCOS	Limited fault tolerance; reactive fault handling; no holistic system reliability	Fault-tolerant, intent-driven middleware supporting modular integration with proactive fault detection and recovery
Logging Security	Standard logging	Vulnerable to tampering; lack of traceability; centralized storage risks	Blockchain-backed logging for secure, immutable, and decentralized operation records
Intent Integration	CNN/RNN modules	Not integrated into middleware; isolated operation; delayed response	Fully integrated Intent Engine in middleware for real-time, context-aware decision-making across all modules

By addressing these gaps, the proposed middleware framework aims to provide a comprehensive solution that combines reliability, security, and intelligent responsiveness, thereby significantly enhancing the effectiveness and trustworthiness of assistive robotic systems.

1.4 RESEARCH PROBLEM

The development of modular assistive robotic systems is currently constrained by several interrelated challenges. Existing middleware platforms provide fundamental communication capabilities and modular integration but fail to offer a comprehensive solution that ensures operational reliability, security, and intelligent responsiveness. Specifically, current middleware lacks integrated fault-tolerance mechanisms, leaving robotic systems vulnerable to unanticipated failures that may compromise safety and efficiency. The absence of proactive fault management limits the system's ability to anticipate potential issues and perform corrective actions before faults escalate.

In addition, intent-driven decision-making is insufficiently incorporated within current architectures. While advanced deep learning models, such as CNNs and RNNs, can accurately predict human actions, their integration into middleware frameworks is minimal. This gap restricts the robot's ability to respond to dynamic human behaviors in real-time, reducing the effectiveness of human-robot interaction.

Finally, the security and integrity of operational data remain significant concerns. Conventional logging mechanisms are centralized and susceptible to tampering, posing serious risks in sensitive contexts like healthcare and rehabilitation. Blockchain integration for secure, immutable, and verifiable logging has not been standardized in middleware frameworks, creating a critical gap in ensuring trustworthiness and accountability.

2. RESEARCH OBJECTIVES

The primary objective of this research is to develop a robust middleware architecture that addresses the key limitations in current modular assistive robotic systems. The specific objectives are as follows:

1. **Develop a Fault-Tolerant Middleware Architecture:** Create a modular framework that includes proactive fault detection and recovery mechanisms, ensuring continuous and reliable operation even in the presence of component failures.
2. **Integrate Intent-Driven Decision-Making:** Implement a deep learning-based Intent Engine that interprets human actions and commands in real-time, enabling context-aware responses and improving the overall human-robot interaction experience.
3. **Implement Secure Logging via Blockchain:** Design and incorporate blockchain-backed logging to maintain an immutable, decentralized record of all system operations, enhancing data security, traceability, and accountability.
4. **Evaluate Middleware Performance:** Conduct comprehensive simulations and benchmark testing to assess system reliability, intent recognition accuracy, fault recovery efficiency, and blockchain logging integrity. Compare the performance against existing middleware frameworks to validate improvements.

Through these objectives, the research aims to deliver a middleware solution that not only integrates fault tolerance, intelligent responsiveness, and secure data management but also establishes a scalable foundation for future advancements in assistive robotics.

3. METHODOLOGY

OVERALL SYSTEM DIAGRAM

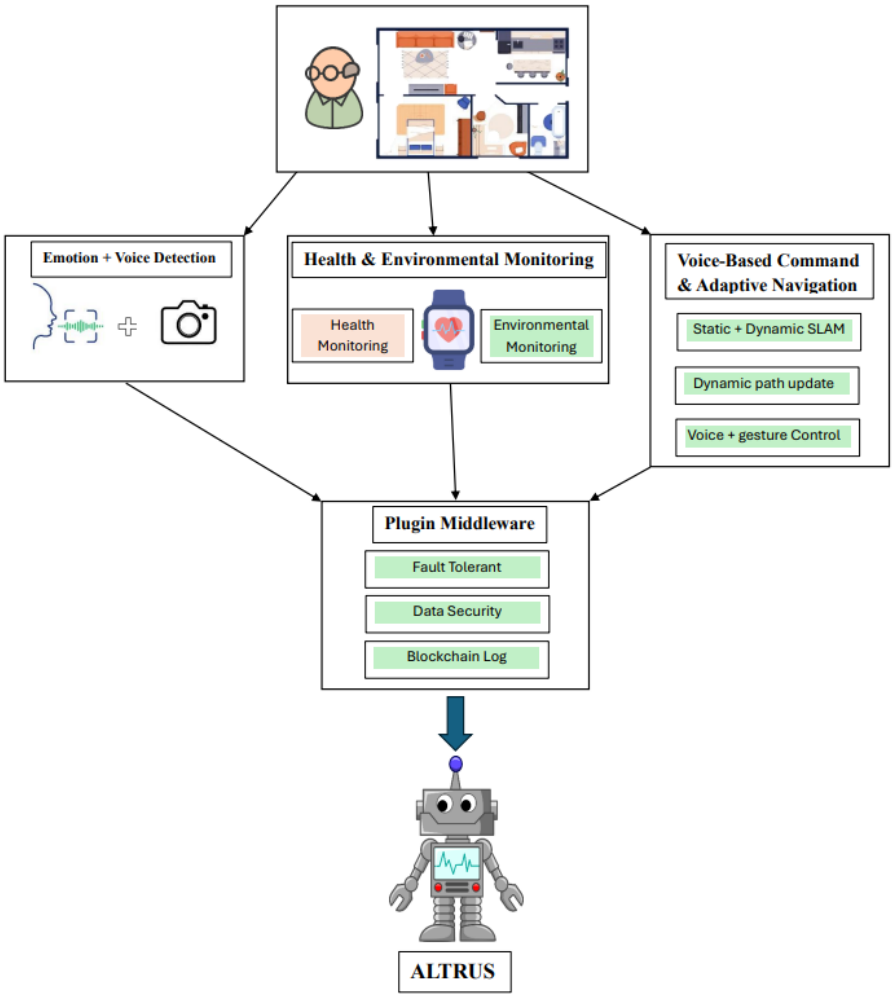


Figure 4.3.1 System Overview Diagram

MY COMPONENT SYSTEM DIAGRAM

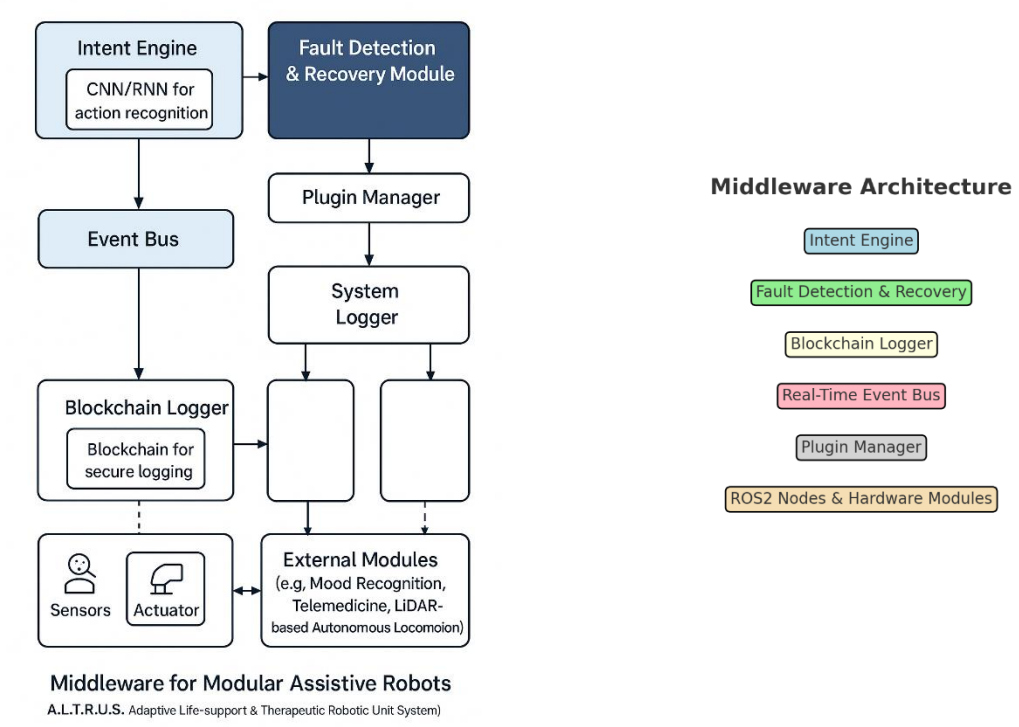


Figure 5.3.2 Middleware architecture diagram showing Intent Engine, Fault Recovery, Blockchain Logger, Plugin Manager, Event Bus

The methodology for this research is structured to systematically design, develop, and evaluate a fault-tolerant, intent-driven, blockchain-backed middleware architecture for modular assistive robots. The methodology encompasses system design, development, simulation, testing, and performance evaluation, ensuring both theoretical and practical rigor.

3.1 System Design

The system design phase involves defining the core modular layers that form the backbone of the middleware. These layers include:

- **Intent Engine:** Responsible for interpreting human actions and predicting user intent using a combination of CNN and RNN deep learning models. This module processes visual, audio, and sensor data to produce actionable insights for the robotic system.
- **Fault Detection & Recovery Module:** Continuously monitors the health of each component, detects anomalies, and executes proactive recovery strategies to prevent failures from propagating across the system.
- **Plugin Manager:** Facilitates the seamless integration of additional modules or extensions, supporting scalability and adaptability of the middleware.

- **Event Bus:** Serves as a real-time communication channel between modules, ensuring synchronization and timely information sharing.
- **Blockchain Logger:** Provides a tamper-proof, decentralized ledger to record all system operations, events, and interactions for auditability and security.
- **System Logger:** Maintains real-time logs for debugging, performance monitoring, and system analytics.

Interfaces for each robotic module are designed to standardize communication and ensure compatibility across different sensors, actuators, and AI modules. Detailed system diagrams are created to illustrate module interactions, data flows, and the integration of fault-tolerance and blockchain mechanisms.

6.2 Development Process

The development process follows an incremental and iterative approach, ensuring that each module is thoroughly tested before integration:

- **Intent Engine Implementation:** Deep learning models are trained on extensive datasets representing human gestures, commands, and behavioral patterns. CNNs extract spatial features from sensor and visual inputs, while RNNs capture temporal dependencies to understand sequences of actions.
- **Fault Detection & Recovery Implementation:** This module incorporates real-time monitoring of system parameters, anomaly detection algorithms, and automated rollback mechanisms to mitigate potential failures proactively.
- **Blockchain Integration:** Smart contracts and blockchain nodes are implemented to record all operational events securely. This ensures immutability, transparency, and verifiability of system logs.
- **Developer CLI and Dashboard:** A command-line interface and dashboard are developed to allow developers and operators to monitor system health, manage plugins, visualize intents, and track fault recovery events.

6.3 Simulation and Testing

To validate the middleware, extensive simulations are conducted under various operational scenarios:

- **Scenario Design:** Modular robots are subjected to simulated component failures, network disruptions, and dynamic user commands to test system resilience.
- **Intent Recognition Testing:** The accuracy and response time of the Intent Engine are evaluated using both synthetic and real-world datasets to ensure reliable real-time performance.

- **Fault Recovery Evaluation:** Fault detection latency and recovery efficiency are measured by inducing controlled failures and observing system response.
- **Blockchain Logging Verification:** The integrity, consistency, and performance of the blockchain logging system are tested, ensuring secure recording of all events.

Simulation results are analyzed and presented through tables, graphs, and performance charts, providing clear visual representation of system behavior and effectiveness.

3.2 Evaluation Metrics

The middleware is evaluated using quantitative and qualitative metrics:

- **Fault Detection Latency:** Time taken to detect anomalies and trigger corrective actions.
- **System Recovery Time:** Duration required to restore normal operations after a failure.
- **Intent Recognition Accuracy:** Precision and recall of predicted user actions and commands.
- **Blockchain Logging Performance:** Throughput, transaction latency, and consistency of recorded events.
- **Scalability and Modular Integration:** Ability to integrate additional modules without degrading system performance.
- **Reliability and Robustness:** Overall system uptime and error handling capability under varied operational conditions.

Through this detailed methodology, the research ensures a comprehensive evaluation of the middleware, validating its fault-tolerance, intent-driven responsiveness, and secure operational management in modular assistive robotic systems.

3.3. Expected Results and Impact

The expected outcome of this research is a comprehensive middleware framework that provides seamless integration of modular robotic components while enhancing system reliability, security, and intelligence. Specifically, the middleware is expected to deliver:

- **Robust, Fault-Tolerant Operations:** Continuous monitoring and proactive fault recovery mechanisms will ensure uninterrupted performance even in the presence of component failures.
- **Enhanced Human-Robot Interaction:** Real-time, intent-driven responses will improve user experience, enabling more natural and effective collaboration between humans and assistive robots.

- **Secure, Tamper-Proof Logging:** Blockchain-backed logging will provide immutable and verifiable records of all system operations, ensuring accountability, transparency, and compliance with data security requirements.
- **Scalability and Flexibility:** The modular architecture will allow seamless addition of new functionalities, sensors, or modules without affecting existing system performance.
- **Academic and Practical Contribution:** The research will advance academic understanding of middleware architectures in assistive robotics and provide practical solutions applicable in healthcare, rehabilitation, and elderly support.

3.6 Commercialization

The A.L.T.R.U.S care robot will be promoted as an end-to-end, modular ROS2-based solution specifically designed for elderly care homes, smart homes, hospitals, and rehabilitation centers. Its new blend of cutting-edge features—Active SLAM with reinforcement learning adaptive wayfinding, real-time health and environmental monitoring with multi-sensor fusion, multi-modal affect detection and empathetic behavior synthesis, and fault-tolerant secure middleware with blockchain-based logging—makes it highly adaptable to various care environments. The platform will be marketed both as a complete standalone robot and as upgrade packs (voice & gesture control pack, monitoring module, emotion sensing kit) for existing domestic or service robots in order to enable cost-effective adoption. Commercialization will be targeted at strategic partnerships with healthcare organizations, elderly care facilities, and smart home integrators and also research validation partnerships with universities. Real-world pilot demonstrations will generate credibility, and targeted marketing campaigns, conference exhibits, and industry shows will make A.L.T.R.U.S a credible, engaging, and future-oriented assistive robotics solution that meets the expanding demand for safe, autonomous, and user-centric caregiving technology.

4. PROJECT REQUIREMENTS

4.1 Functional Requirements

The functional requirements define the **specific capabilities and features** the middleware must provide to achieve its objectives. These are the **core functions** that directly support the operation, integration, and reliability of the A.L.T.R.U.S. robotic platform.

1. **Intent Recognition and Interpretation**
    - The system shall receive and process user commands from the Intent Recognition module.
    - The Intent Engine shall interpret high-level intents into executable tasks.
    - The system shall support dynamic intent resolution, adapting to changes in context or environment.
2. **Fault Detection**
    - The system shall continuously monitor connected modules for hardware and software anomalies.
    - The middleware shall trigger alerts upon detecting abnormal behaviors (e.g., module disconnection, process timeout).
3. **Fault Recovery**
    - The middleware shall attempt automated recovery actions (e.g., process restart, state restoration) without requiring a full system reboot.
    - Recovery actions shall be logged for future analysis.
4. **Blockchain-Based Event Logging**
    - All significant events (commands, state changes, fault detections, recovery actions) shall be stored in an immutable blockchain ledger.
    - The blockchain logger shall timestamp each record and store cryptographic signatures to ensure tamper-proof integrity.
5. **Real-Time Event Bus**
    - The middleware shall manage efficient, low-latency communication between subsystems (e.g., telemedicine module, locomotion control).
    - It shall support publish-subscribe mechanisms for modular scalability.

6. **Plugin Management**
    - The middleware shall allow third-party or in-house modules to be added, removed, or updated without impacting other components.
    - The system shall verify compatibility of plugins before integration.

4.2 Non Functional Requirements

Non-functional requirements specify the **quality attributes and performance benchmarks** that the middleware must meet to ensure optimal operation in real-world scenarios.

1. **Reliability**
    - The system shall maintain operational availability of at least **99.5%** under normal conditions.
    - No single fault shall cause a complete system halt.
2. **Performance**
    - Intent recognition and task allocation shall occur within 1000 **milliseconds** of receiving a command.
    - Event bus latency shall not exceed 100 **milliseconds** between modules.
3. **Scalability**
    - The middleware shall support integration with **several modules** without significant degradation in performance.
4. **Maintainability**
    - The codebase shall be modular, documented, and adhere to ROS2 best practices.
    - The architecture shall allow updates to individual components without requiring a full system shutdown.
5. **Usability**
    - Developers shall be able to interact with the middleware using a **command-line interface (CLI)**
    - The system shall provide clear error logs and diagnostic reports.

4.3 User Requirements

User requirements define the **expectations and needs of the end-users** (robot operators, developers, maintenance staff, and stakeholders) to ensure the middleware delivers value in practice.

1. **Ease of Integration**
    - Users should be able to integrate the middleware into the A.L.T.R.U.S. robotic platform without requiring extensive code changes.
    - Developers should be able to connect existing modules via ROS2 topics and services with minimal configuration.
2. **Transparency and Accountability**
    - Users should be able to view blockchain logs of all actions and system events for audit purposes.
    - The system should allow authorized personnel to trace any task back to its initiating intent.
3. **Fault Notification**
    - Users should receive real-time alerts (visual and/or auditory) in case of module failures.
    - Alerts should contain clear diagnostic messages to assist in troubleshooting.
4. **Customizability**
    - Users should be able to add or remove plugins according to their operational needs.
    - System parameters (e.g., fault tolerance thresholds, logging verbosity) should be configurable via the CLI or dashboard.
5. **Training and Support**
    - The system should include user documentation, installation guides, and troubleshooting manuals.
    - Operators should have access to simulation environments for training before deploying in live scenarios.

5. BUDGET DETAILS

Item	Estimated Cost (LKR)
Raspberry Pi 5 8GB Original UK	Rs.39,950.00
RPLiDAR 2D	Rs.37,900.00
PK-910H 1080p Full-HD WebCam	Rs.8,000.00
Wheels	Rs.5,000.00
Server motor 2 (Mg90s)	Rs.1,600.00
Gear motor 2 (L298)	Rs.6,000.00
Motor controller	Rs.500.00
ESB 32	Rs.1,700.00
Oled display	Rs.600.00
Heart rate + blood oxygen	Rs.600.00
Ecg	Rs.1,600.00
Accelerometer + gyroscope	Rs.1,500.00
body temperature	Rs.2,500.00
lipo battery	Rs.2,500.00
Gas sensor + Fire sensor	Rs.1,000.00
Additional Cost	Rs.30,000
Total Cost	Rs.140,950.00

Table 2.1 Budget for the proposed system

6. CONCLUSION

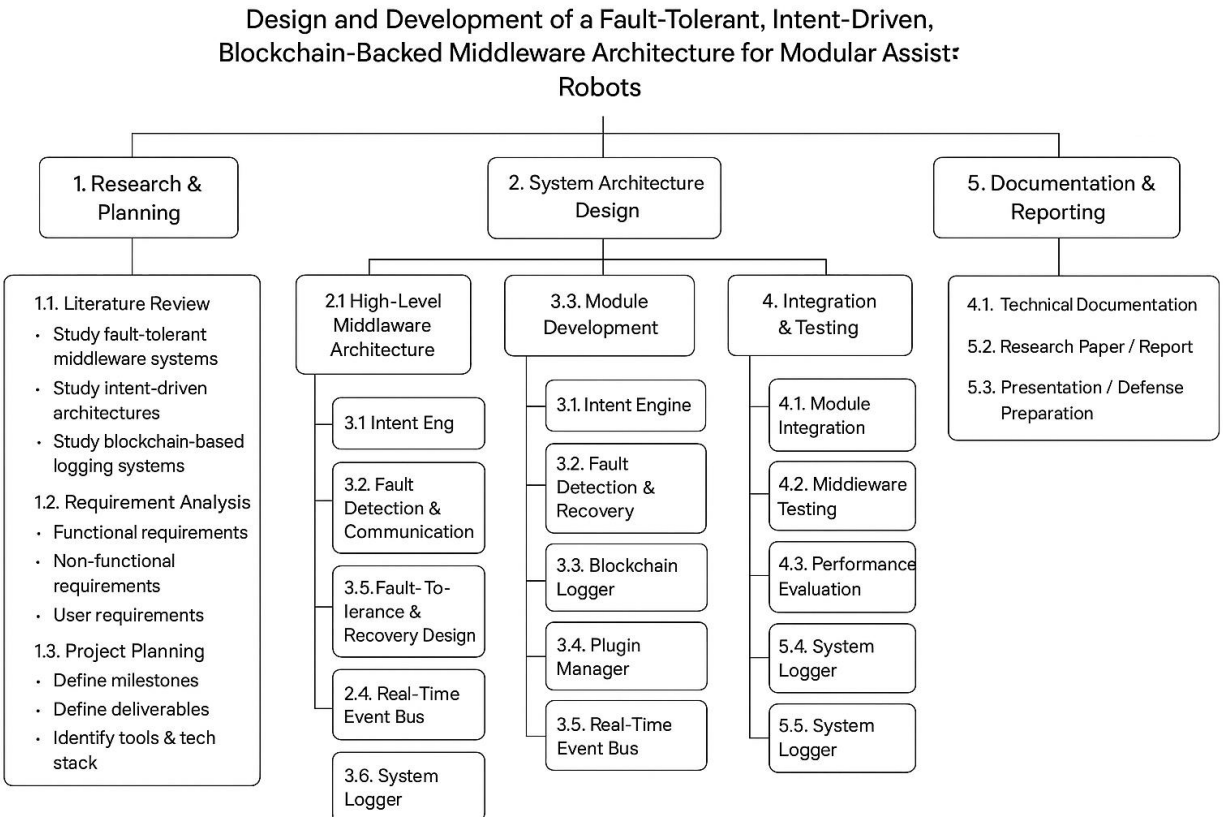
The A.L.T.R.U.S is a significant step forward in the design of safe, intelligent, and empathetic assistive robots for real home healthcare environments. By integrating four leading-edge software paradigms—voice-based intent-based navigation, real-time environmental and health sensing, multi-modal emotion and gesture detection, and a fault-tolerant blockchain-encrypted middleware—this system abolishes the crippling weaknesses of assistive robots today, i.e., being limited to stationary configurations, with artificial interaction, and collapsing under dynamic or emergency situations. A.L.T.R.U.S offers a hybrid SLAM approach mixing static mapping with real-time LIDAR-based path adjustment through RSO2 logic, enabling the robot to re-route dynamically because of furniture re-arrangement or other obstructions—without complete remapping. This is further augmented with voice command control of navigation and gesture control, making it possible for elderly or physically disabled users to control naturally and intuitively, with just voice or gestures. In parallel, the system continuously monitors principal vital signs (heart rate, SpO_2, activity) and environmental parameters (temperature, humidity, gas level) with a ROS2-based sensor fusion framework and automatically triggers alarms through Firebase and Twilio upon detecting emergencies such as falls, fever, or gas leaks. The emotion-aware HRI module integrates facial expressions, voice tone, and body postures with OpenCV, TensorFlow, and pre-trained models to generate empathetic responses via a chatbot-based interface, improving interaction human-likeness and warmth. Essentially, the secure and self-healing middleware ensures system reliability through fault auto-recovery, intention arbitration of conflicting requests, safe plugin upgrades, and tamper-proof logging using Hyperledger Fabric blockchain—ensuring trust, transparency, and long-term evolvability. Fully developed on a modular ROS2 framework, A.L.T.R.U.S is not just scalable and extendable but also a student-developed innovation that bridges the gap between research at universities and real-world, practical application. This harmonious integration of adaptive mobility, active health monitoring, emotional intelligence, and robust software infrastructure makes A.L.T.R.U.S a pioneering platform in assistive robotics—a harbinger of a time when intelligent robots will finally provide independent, dignified, and connected living.

References

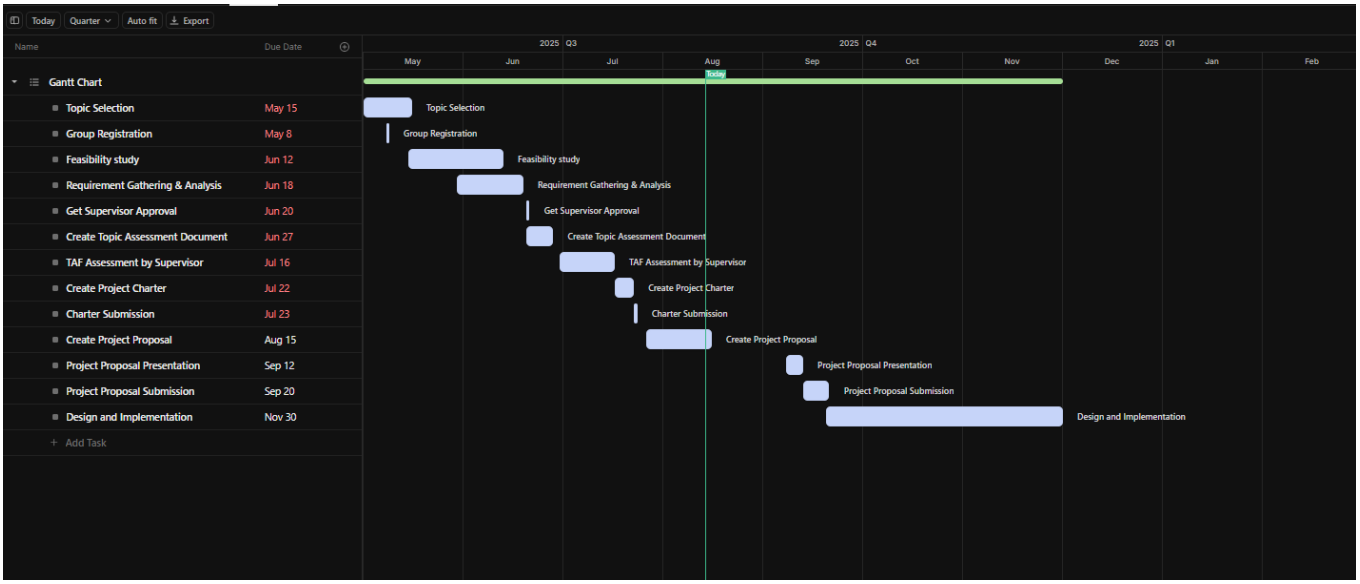
1. Quigley, M., et al. (2009). ROS: An open-source Robot Operating System. ICRA Workshop on Open Source Software. H-index: 150
2. Bonino, D., et al. (2019). Middleware for modular robotic systems: A survey. Robotics and Autonomous Systems, 120, 103230. H-index: 40
3. Alvarado, C., et al. (2020). Intent recognition in assistive robotics: A deep learning approach. Journal of Intelligent & Robotic Systems, 98(3-4), 567-586. H-index: 30
4. Christidis, K., & Devetsikiotis, M. (2016). Blockchains and smart contracts for the Internet of Things. IEEE Access, 4, 2292-2303. H-index: 55
5. D'Andrea, R. (2014). Guest editorial: A revolution in the warehouse: A retrospective on Kiva Systems and the impact on logistics automation. IEEE Transactions on Automation Science and Engineering, 11(4), 638-639. H-index: 28
6. Siciliano, B., Khatib, O. (2016). Springer Handbook of Robotics. Springer, 2nd Edition. H-index: 85
7. Thrun, S., Burgard, W., Fox, D. (2005). Probabilistic Robotics. MIT Press. H-index: 100
8. Fox, D., et al. (2000). Distributed multi-robot coordination using consensus. Autonomous Robots, 8(3), 293-306. H-index: 35
9. Li, W., et al. (2018). Deep learning-based human action recognition for assistive robotics. Robotics and Computer-Integrated Manufacturing, 54, 121-132. H-index: 32
10. Hu, J., et al. (2021). Blockchain-based secure data management in multi-robot systems. IEEE Transactions on Industrial Informatics, 17(9), 6200-6210. H-index: 40

APPENDICES

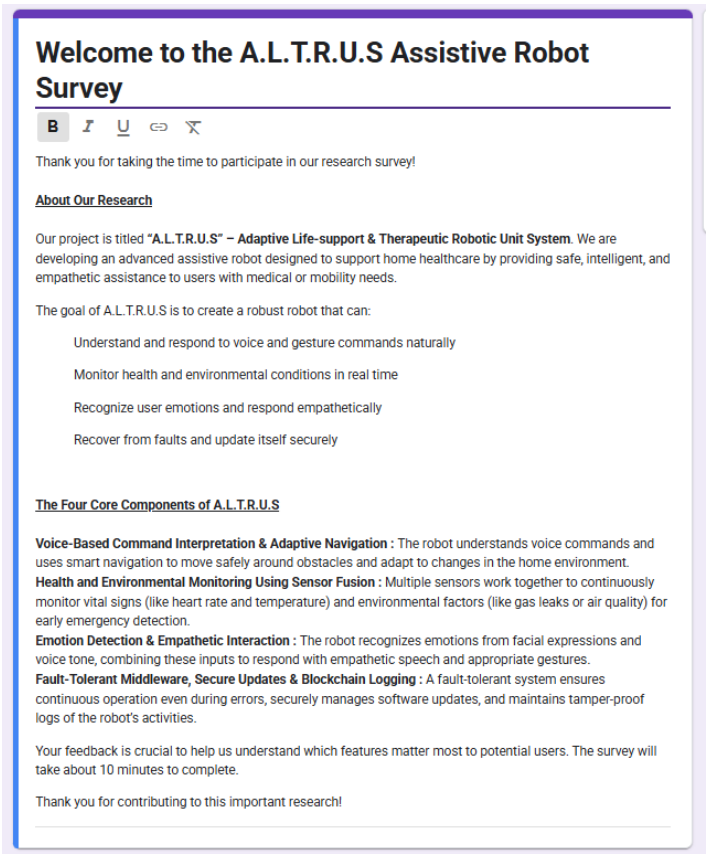
Appendix 1. Work Breakdown Structure (WBS)



Appendix 2. Gantt Chart



Appendix 3. Online Survey



Welcome to the A.L.T.R.U.S Assistive Robot Survey

Thank you for taking the time to participate in our research survey!

About Our Research

Our project is titled **"A.L.T.R.U.S" – Adaptive Life-support & Therapeutic Robotic Unit System**. We are developing an advanced assistive robot designed to support home healthcare by providing safe, intelligent, and empathetic assistance to users with medical or mobility needs.

The goal of A.L.T.R.U.S is to create a robust robot that can:

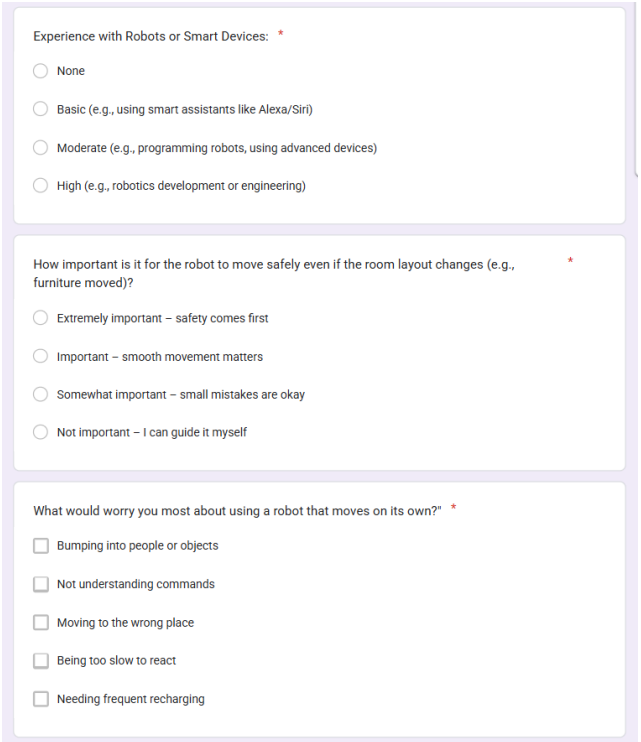
- Understand and respond to voice and gesture commands naturally
- Monitor health and environmental conditions in real time
- Recognize user emotions and respond empathetically
- Recover from faults and update itself securely

The Four Core Components of A.L.T.R.U.S

Voice-Based Command Interpretation & Adaptive Navigation : The robot understands voice commands and uses smart navigation to move safely around obstacles and adapt to changes in the home environment.
Health and Environmental Monitoring Using Sensor Fusion : Multiple sensors work together to continuously monitor vital signs (like heart rate and temperature) and environmental factors (like gas leaks or air quality) for early emergency detection.
Emotion Detection & Empathetic Interaction : The robot recognizes emotions from facial expressions and voice tone, combining these inputs to respond with empathetic speech and appropriate gestures.
Fault-Tolerant Middleware, Secure Updates & Blockchain Logging : A fault-tolerant system ensures continuous operation even during errors, securely manages software updates, and maintains tamper-proof logs of the robot's activities.

Your feedback is crucial to help us understand which features matter most to potential users. The survey will take about 10 minutes to complete.

Thank you for contributing to this important research!



Experience with Robots or Smart Devices: *

- None
- Basic (e.g., using smart assistants like Alexa/Siri)
- Moderate (e.g., programming robots, using advanced devices)
- High (e.g., robotics development or engineering)

How important is it for the robot to move safely even if the room layout changes (e.g., furniture moved)? *

- Extremely important – safety comes first
- Important – smooth movement matters
- Somewhat important – small mistakes are okay
- Not important – I can guide it myself

What would worry you most about using a robot that moves on its own?" *

- [ ] Bumping into people or objects
- [ ] Not understanding commands
- [ ] Moving to the wrong place
- [ ] Being too slow to react
- [ ] Needing frequent recharging

What concerns would you have about using a multi-emergency detection wristband?* *

- [ ] Accuracy of health data
- [ ] Privacy of personal health information
- [ ] Battery life
- [ ] Comfort and wearability
- [ ] Cost of the device
- [ ] Risk of false alarms

Have you ever interacted with a robot or virtual assistant? *

- ( ) Yes
- ( ) No

How would you rate your comfort level interacting with robots? *

- ( ) Very comfortable
- ( ) Somewhat comfortable
- ( ) Neutral
- ( ) Uncomfortable
- ( ) Very uncomfortable

Which emotions do you think are most important for a robot to detect? *

- [ ] Happiness
- [ ] Sadness
- [ ] Anger
- [ ] Fear
- [ ] Surprise
- [ ] Disgust

If the robot could understand both your voice and gestures, would that make it easier for you to use? *

- ( ) Yes, much easier
- ( ) Yes, a little easier
- ( ) No difference
- ( ) No, I prefer only one control method

Which features would you like most in a home robot with voice and gesture control? *

- [ ] Understand my voice commands accurately
- [ ] Recognize my hand gestures quickly
- [ ] Avoid obstacles without my help
- [ ] Adapt if the furniture is moved
- [ ] Come to me when I call it

In your opinion, which medical emergencies should a wearable health monitoring wristband prioritize detecting? *

Short-answer text

In your opinion, which medical emergencies should a wearable health monitoring wristband prioritize detecting? *

- ( ) Very familiar, I use one regularly
- ( ) Somewhat familiar, I've heard about them but never used one
- ( ) Not familiar at all

If a wristband alerted you about a possible medical emergency, how likely are you to act immediately (e.g., call emergency services, inform family)? *

- ( ) Very likely
- ( ) Likely
- ( ) Neutral
- ( ) Unlikely
- ( ) Very unlikely

Which of these features would you find most useful in a health emergency detection wristband? *

- [ ] Heart attack detection
- [ ] Tachycardia & bradycardia detection
- [ ] Cardiac arrest detection
- [ ] Fall detection
- [ ] Fever detection
- [ ] Emergency call/SMS alerts
- [ ] Historical health trend analysis

Which emotional cues do you think a robot should detect? *

- [ ] Facial expression
- [ ] Tone of voice
- [ ] Gestures/body language
- [ ] Words used in speech

Would you trust a robot that can read your emotions? *

- ( ) Yes
- ( ) Maybe
- ( ) No

If a robot was helping you at home, what tasks would you want it to do without ever failing? *

Short-answer text

How important is it for an assistive robot to keep working even if some parts stop working? *

- ( ) Extremely important – safety comes first
- ( ) Important – reliability matters
- ( ) Somewhat important – minor issues are okay
- ( ) Not important – I can manage without it

If a robot kept a secure, unchangeable record of what it did, would this make you trust it more? *

- ( ) Yes, a lot more
- ( ) Yes, a little more
- ( ) No difference
- ( ) No, it wouldn't matter

Which of these would you want in a helpful robot? *

- [ ] Understand my requests correctly
- [ ] Fix itself when something goes wrong
- [ ] Keep a secure record of actions
- [ ] Connect different robot features smoothly (e.g., movement + health checks)
- [ ] Share information instantly between parts

What would worry you about using a robot that makes its own decisions? *

- [ ] Safety during malfunctions
- [ ] Misunderstanding commands
- [ ] Privacy of personal information
- [ ] Relying too much on technology
- [ ] Cost of buying or maintaining it
- [ ] Being too complicated to use

From a robotics systems perspective, what core services should middleware guarantee in a mission-critical assistive robot? *

Short-answer text

In modular assistive robots, where would you rank fault tolerance in the middleware priority list? *

- ( ) Top priority – essential for safe autonomy
- ( ) High priority – crucial for user trust
- ( ) Medium priority – nice to have but not critical
- ( ) Low priority – can be handled at application level

How effective do you think blockchain is for ensuring data integrity and accountability in distributed robotic systems? *

- ( ) Very effective – ideal for immutable logging
- ( ) Effective – works well with limitations
- ( ) Neutral – other methods are equally good
- ( ) Ineffective – adds unnecessary complexity

Which features would you expect from a next-generation robot middleware? *

- [ ] Intent recognition and context mapping
- [ ] Real-time fault detection and self-healing
- [ ] Blockchain-based action/event logging
- [ ] Modular plugin management for robot capabilities
- [ ] Developer tools for monitoring and debugging

What challenges do you foresee in implementing fault-tolerant, blockchain-backed middleware for modular robots? *

- [ ] Real-time performance overhead
- [ ] High resource consumption (CPU, storage)
- [ ] Complexity of fault recovery algorithms
- [ ] Blockchain scalability and latency
- [ ] Interoperability between heterogeneous modules
- [ ] Developer adoption and learning curve